

Accelerated Preference Elicitation with LLM-Based Proxies

David Huang ^{*} Francisco Marmolejo-Cossío [†] Edwin Lock [‡] David Parkes [§]

October 30, 2024

Abstract

Elicitation in combinatorial auctions is challenging as bidder preferences may be inherently difficult to describe and consequently communicate to an auctioneer. Classical work in elicitation focuses on using query-based techniques inspired from proper learning—often via *proxies* which interface between bidders and a core auction mechanism—to incrementally learn bidder preferences as needed to compute efficient allocations. Although such elicitation mechanisms enjoy theoretical query efficiency, the amount of communication required may still be too cognitively taxing for bidders in practical scenarios.

Significant recent advancements in natural language processing, particularly Large Language Models (LLMs), suggest the use of natural language for eliciting preferences. In this paper, we propose an efficient LLM-based proxy design for eliciting preferences from bidders in a combinatorial auction setting where communication is intentionally limited. Our proposed mechanism combines LLM pipelines and DNF-proper-learning techniques to quickly approximate preferences with bounded communication overhead. To validate our approach, we create a broader testing sandbox for elicitation mechanisms that make use of natural language as a means of communication.

The LLM-based approach reaches approximately efficient outcomes five times faster than classical proper learning based elicitation mechanisms, demonstrating the potential power of natural language based elicitation approaches. Moreover, the LLM-based proxies provide outcomes that converge, with enough communication, to those arising from DNF-proper-learning techniques.

1 Introduction

In recent years, the field of auction design has seen significant advancements, driven by the increasing complexity of market environments and the need for more sophisticated mechanisms to handle diverse and multi-dimensional preferences. Auctions are fundamental tools in both theoretical and practical economics, enabling efficient allocation of resources across a variety of domains, from spectrum sales Cramton [1997] to procurement Klemperer [2004] and beyond Palacios-Huerta et al. [2024].

Traditional auction mechanisms place a high burden on the bidder. For a bidder, giving a full articulation of their preferences in direct revelation mechanisms can place impractical cognitive demands Budish and Kessler [2022] and can quickly become prohibitive to even express for various valuation classes Nisan and Segal [2006]. Work on preference elicitation aims to address these issues by having bidders incrementally reveal their preferences, where bidders only communicate the information necessary to resolve the auction. Instead of needing to produce the entirety of their preferences bidders only need to produce comparatively simpler responses to queries. In this context, the query complexity of elicitation mechanisms can be polynomial in the size of the description of a valuation Parkes [2005]. However, even a polynomial query complexity can be prohibitive in practice, as bidders may require a non-trivial amount of time in answering queries Ergin and Sarver [2010], Nielsen and Rehbeck [2022]. Moreover, this problem becomes increasingly difficult as economic environments become more complex Braziunas [2006].

In response to these challenges, our research introduces a novel approach that leverages natural language to communicate preferences. After all, what would be a more natural choice? In this paper, we

^{*}Harvard University, dhuang@college.harvard.edu

[†]Harvard University, fjmarmol@g.harvard.edu

[‡]University of Oxford, edwin.lock@cs.ox.ac.uk

[§]Harvard University, parkes@eecs.harvard.edu

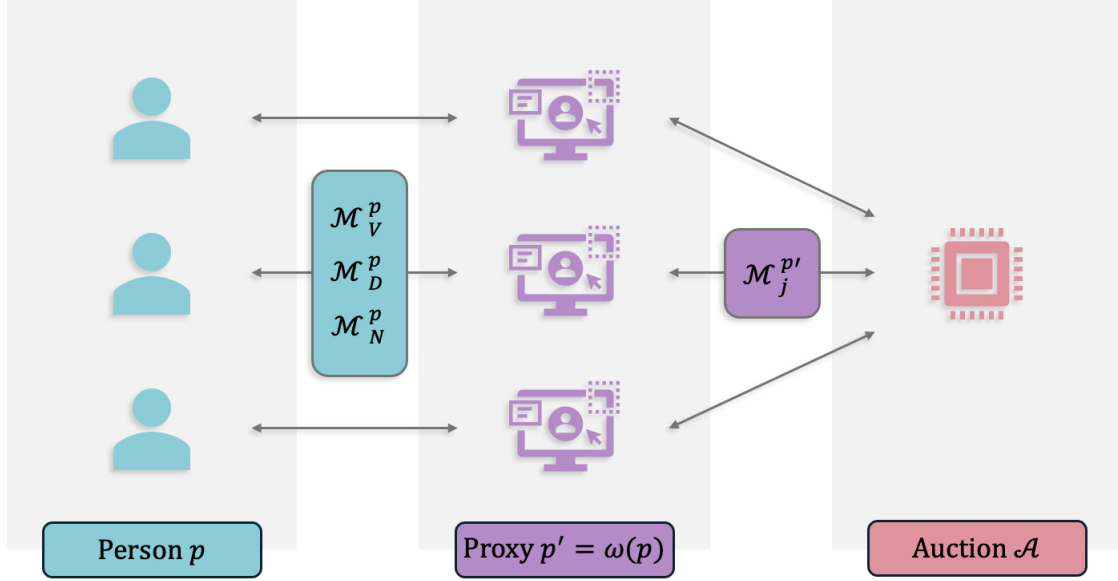


Figure 1: Diagram of the relevant components. There are three categories of actors: people, proxies, and the auction. Each of the actors are completely distinct from each other and can only communicate via messaging by calling $\mathcal{M}$. The proxies mediate the communication between the people and the auction. Proxies communicate with people by messaging them with value queries $\mathcal{M}_V^p$, demand queries $\mathcal{M}_D^p$, and natural language questions $\mathcal{M}_N^p$. Auctions communicate with proxies by messaging them with messages of type j by calling $\mathcal{M}_j^{p'}$ where the specific message type(s) depend on the auction design.

use LLMs to enhance the efficiency and practicality of preference elicitation in auctions. By integrating LLMs into the framework of incremental revelation mechanisms, specifically Lahaie and Parkes’s competitive equilibrium combinatorial auction (CECA) Lahaie and Parkes [2004], we propose a system that shows promise in modeling and inferring bidder preferences more effectively than traditional methods. In our approach, LLMs (depicted in the middle, purple panel of Figure 1) are utilized as proxies in the elicitation process, where a proxy acts as an intermediary that interacts with the auction mechanism on behalf of the bidder, querying the bidder to uncover preferences in a more intuitive and less cognitively demanding way through natural language interactions.

To empirically test the effectiveness of proxies in our auction design, we develop a comprehensive pipeline that simulates the bidding process across multiple items. The pipeline utilizes the ability of LLMs to simulate human bidder behavior Horton [2023] to capture the complexities and limitations that motivated our study of elicitation — specifically, the challenges that arise when preferences are inconsistent with a mathematical valuation function or when bidders struggle to respond to repeated queries about their preferences. In our approach, bidder “types” are dictated by a seed description in natural language that encapsulates their preferences, and an LLM (distinct from the proxy LLM) uses this seed to generate responses to queries, simulating the behavior of bidders with such characteristics. The LLM proxy (depicted in the middle, purple panel of Figure 1) then interacts with these bidder LLMs (depicted in the middle, red panel of Figure 1), querying them to learn about their underlying preferences, and engages with the CECA Lahaie and Parkes [2004] — to represent the bidders in the bidding process.

After the auction concludes, we evaluate the social welfare by summing the values each simulated bidder ascribes to their allocated bundle and comparing this to the welfare-maximizing outcome given full knowledge of the simulated bidder’s implied XOR valuation. This approach allows us to measure not only the accuracy and efficiency of preference elicitation but also the overall effectiveness of LLMs serving as proxies within a real-world-inspired auction setting.

We show that the seeds are robust by showing that different LLMs give similar valuations for the same bundle when shown the same seed. We benchmark the performance of the LLM proxy against

proxies that utilize traditional proper learning methods Lahaie and Parkes [2004] to infer LLM bidder preferences. To thoroughly evaluate these proxies, we vary the number of queries they are permitted to make to bidders, assessing both how well they learn bidder preferences and the overall efficiency of the allocation within the specified auction framework. The results of our study are encouraging: LLM proxies demonstrate the ability to learn bidder preferences more rapidly and with five times fewer queries than traditional methods, leading to lower errors in the representation of these preferences. Furthermore, the welfare outcomes achieved when LLMs act as proxies indicate that these models are not only feasible but also highly effective in facilitating efficient auction designs. Our findings suggest that integrating LLMs into auction mechanisms presents a promising avenue for enhancing both the theoretical and practical aspects of auction design, especially in environments where traditional elicitation methods may fall short.

2 Preliminaries

We consider a variety of *auction scenarios* with different types of items. A scenario $\mathcal{S}$ has N items $\mathcal{T} = \{T_1, \dots, T_N\}$. Moreover, we have a population $\mathcal{P}$ of *people*, where $p \in \mathcal{P}$ is a specific person participating in the auction. A *bundle* b of items is a subset of $\mathcal{T}$. A *price function* $\phi : \mathcal{B} \rightarrow \mathbb{R}_+$ assigns a (uniform) price to each item.

We assume that the objects of an auction scenario – the scenario itself, its items, bundles, prices, and people – have a natural language description given by the *description function* $\mathcal{D}$. For instance, we may have $\mathcal{D}(\mathcal{S}) = \text{“a farmer’s market”}$ and three types of items, $\mathcal{D}(T_1) = \text{“apple”}$, $\mathcal{D}(T_2) = \text{“banana”}$ and $\mathcal{D}(T_3) = \text{“strawberry”}$. In addition, we might have a person with a description given by $\mathcal{D}(p) = \text{“Carlos; only likes red fruit”}$ bidding in the auction. For expedience, we often directly refer to an object by its description.

People in a scenario participate in the auction via a proxy, which makes a finite number of queries to the person. We consider a family Ω of proxy designs. A person p is represented by a proxy $p' = \omega(p)$ with $\omega \in \Omega$. This proxy has an internal context, or ‘memory’, and so can refine its understanding of the person’s preferences by repeatedly communicating with its person. Similarly, each person has a memory of the conversations between it and its proxy.¹ A person communicates with its proxy by responding to natural language questions, value queries, and demand queries made by the proxy. We write $\mathcal{M}_N^p$ (Natural Language Query) for the function that maps any natural language question to person p ’s answer. Similarly, $\mathcal{M}_V^p$ (Value Query) maps any bundle to a (non-negative real) value that person p has for the bundle.² Finally, $\mathcal{M}_D^p$ (Demand Query) takes prices ϕ and a bundle b as input and returns either $(1, b)$ if the person is happy with b at ϕ or $(0, b')$ if the person would prefer some bundle $b' \neq b$ over b at ϕ . After each of these questions, the question and answer form part of the person and proxy’s memories.

When evaluating our LLM proxy design, we simulate people by secondary LLMs (distinct from the LLM proxy). These LLMs are seeded with the description $\mathcal{D}(p)$ of virtual people. In this setting, we assume that the functions $\mathcal{M}_N^p$, $\mathcal{M}_V^p$, and $\mathcal{M}_D^p$ are implemented by the secondary LLM instead of a person directly.

3 The Auction Framework

We explore the potential of LLM proxies in the competitive equilibrium combinatorial auction and use an XOR-preference-elicitation algorithm and proxy serve as the gold standard against which we compare our LLM-based proxies Lahaie and Parkes [2004].

3.1 XOR preference elicitation

An XOR preference $\theta = (B, v)$ consists of a set of atomic bundles $B \subseteq \mathcal{B}$ and valuation function $v : B \rightarrow \mathbb{R}_+$. For any bundle $b \in \mathcal{B}$, this preference θ induces the valuation

$$v^*(b) = \max_{\{b' \in B \mid b' \subseteq b\}} v(b')$$

¹This can be formally captured by a context c_i which contains a transcript of the first i (natural language) communications. This is how the context is implemented in our LLM proxy design.

²The implementation of these message functions may not be deterministic.

Algorithm 1 Learn XOR Preferences ($\mathcal{L}_{xor}(p)$)

```
1: Input: Person  $p$ 
2: Initialize  $B \leftarrow \emptyset$ ,  $v \leftarrow \emptyset$ , equivalent  $\leftarrow$  False
3: while not equivalent do
4:    $\phi^* \leftarrow$  valuation.to.induced.prices
5:    $(r, b) \leftarrow \mathcal{M}_D^p(\phi^*, b)$  {Obtain response and bundle from demand query}
6:   if  $r = 1$  then
7:     return  $(B, v)$  {All atomic bundles discovered}
8:   end if
9:    $(b', \text{value}) \leftarrow \mathcal{L}_{xor, step}(p, b)$  {Identify atomic bundle and its value}
10:   $B \leftarrow B \cup \{b'\}$ 
11:   $v(b') \leftarrow \text{value}$ 
12:  if All atomic bundles discovered then
13:    equivalent  $\leftarrow$  True
14:  end if
15: end while
16: return  $(B, v)$ 
```

This formulation captures the idea that the value of a bundle is determined by the highest-valued atomic bundle it contains. Similarly, prices ϕ induce prices ϕ^* , where the price of a bundle is the highest price of a bundle it contains. The induced valuations and prices capture the notion of *free disposal*, i.e., the price or valuation of a bundle do not increase when an item in the bundle is removed. Let **valuation.to.induced.prices** be the algorithm which takes a valuation (B, v) , constructs prices ϕ where $\phi(b) = v(b)$ for all $b \in B$ and returns the induced prices ϕ^* .

The XOR preference elicitation algorithm makes a series of demand and value queries to compute a person's XOR preferences $\theta = (B, v)$. We denote this algorithm as $\mathcal{L}_{xor}$ and formulate it from Lahaie and Parkes [2004], who were inspired by the connection between XOR bids and monotone-DNFs identified by Blum et al. [2004]. We directly describe it in terms of value and demand queries.

$\mathcal{L}_{xor, main}$, described in Algorithm 1, begins with an empty set of atomic bundles B and iteratively discovers new atomic bundles by performing demand queries. For each demand query, if the response indicates that the current valuation correctly represents the person's preferences, the algorithm concludes that all atomic bundles have been identified and terminates. Otherwise, it invokes the $\mathcal{L}_{xor, step}$ subroutine to extract a new atomic bundle from the provided bundle b from the demand query itself.

In the $\mathcal{L}_{xor, step}$ procedure, described in Algorithm 2, the algorithm queries the for value of the entire bundle b from the demand query using $\mathcal{M}_V^p(b)$ and then attempts to identify which items are essential by removing them one at a time and querying the value again using $\mathcal{M}_V^p(b')$. If removing an item does not change the valuation, it implies that the item is not essential to the bundle's value and can be excluded from the atomic bundle. This process is repeated for all items in the bundle to identify the minimal atomic bundle b' , which is then added to the set B along with updating the valuation function to the queried value of b' .

This iterative process continues until all atomic bundles are discovered, ensuring that the algorithm accurately learns the person's XOR preferences with a bounded number of queries.

3.2 Competitive equilibrium combinatorial auction

Given a scenario $\mathcal{S}$ and K people $p_1, \dots, p_K$ represented by their proxies $p'_i = \omega(p_i)$, for a given $\omega \in \Omega$, we define an auction $\mathcal{A}$ as an algorithm that takes as input $\mathcal{S}$ and $p'_1, \dots, p'_K$ and gives as output an allocation $(b_1, \varphi_1), \dots, (b_K, \varphi_K)$ where the person p_i is allocated bundle b_i and must pay cost φ_i . The auction algorithm only communicates with proxies.

We will work with the competitive equilibrium combinatorial auction (CECA), described by Lahaie and Parkes [2004] and which we'll denote $\mathcal{L}_{CECA}$, in which the auction algorithm runs an elicitation procedure on each of the participants via proxies until all the participants signal that they are satisfied, hence reaching a competitive equilibrium outcome. A high-level diagram of the interactions between the auction, proxies, and people are shown in Figure 1. The auction maintains

Algorithm 2 Learn XOR Preferences Step ($\mathcal{L}_{xor,step}(p, b)$)

```
1: Input: Person  $p$ , Bundle  $b$  {Current bundle from demand query}
2:  $\nu \leftarrow \mathcal{M}_V^p(b)$  {Obtain original bundle value}
3:  $b' \leftarrow b$  {Initialize refined bundle}
4: for  $i = 1$  to  $N$  do
5:    $b'' \leftarrow b'$ 
6:    $b''[i] \leftarrow 0$  {Remove item  $i$  from bundle}
7:    $\nu' \leftarrow \mathcal{M}_V^p(b'')$  {Query new bundle value}
8:   if  $\nu' = \nu$  then
9:      $b'[i] \leftarrow 0$  {Item  $i$  is not essential}
10:  end if
11: end for
12: return  $(b', \nu)$  {Return identified atomic bundle and its value}
```

manifest valuations for each person, in our case an XOR valuation $\tilde{v}$, that is used to solve for the allocation via integer programming and that is updated if a person is not satisfied. The goal is to incrementally reveal a manifest valuation such that a competitive equilibrium can be reached.

We characterize a single iteration of the elicitation procedure that the auction calls the proxy to perform as the algorithm $\mathcal{M}_C^{p'}$ (CECA Step) with the inputs to the algorithm being the allocated bundle b and prices ϕ and the output to the algorithm being either $(1, \theta)$, indicating that the person the proxy represents is satisfied and confirms their manifest valuation θ , or $(0, \theta')$, indicating that the person the proxy represents is not satisfied and thus would like to update their manifest valuation to θ' . This procedure is described in detail in Algorithm 3.

3.3 XOR elicitation proxy

The XOR elicitation proxy ω_{xor} maintains in its context the manifest valuation $\theta = (B, v)$ which is initialized to $B = \emptyset$ and $v : \emptyset \rightarrow \mathbb{R}$.

The proxy supports message $\mathcal{M}_C^{p'}$, for $p' = \omega_{xor}(p)$, which it implements as the following algorithm. Given inputs, ϕ, b , it calls a demand query $\mathcal{M}_D^p(\phi, b)$ using the the given prices and allocated bundle. The demand query outputs (r, b') where r is 1 if the person is satisfied and 0 otherwise and b' is the desired bundle. If r is 1 the algorithm returns $(1, \theta)$. Otherwise it calls $\mathcal{L}_{xor,step}(p, b)$, which outputs (B', ν) , and the algorithm uses this to return as output the updated manifest valuation $\theta' = (B + \{b'\}, v'(x))$ where v' is the function that maps b to $v(b)$ for $b \in B$ and maps b' to ν .

3.4 Scenarios

We create three scenarios in which we test the LLM pipelines for modeling bidders and the LLM pipelines for acting as proxies in the competitive equilibrium combinatorial auction. We design the scenarios to be accessible so that the reader can readily assess the realism of the interactions and various LLMs can be tested without domain specific knowledge or fine tuning. Each scenario has $N = 6$ items for auction, where some of items are clear substitutes for and others are clear complements to each other and where many are somewhere in between. These scenarios and their items are presented in Table 1.

Algorithm 3 Competitive Equilibrium Combinatorial Auction ($\mathcal{L}_{CECA}$)

```

1: Input:
2:    $\mathcal{S}$ : Scenario
3:    $p'_1, p'_2, \dots, p'_K$ : Proxies for each person  $p_i$ 
4: Initialize:
5:   Set manifest valuations  $\tilde{\theta}_1, \tilde{\theta}_2, \dots, \tilde{\theta}_K \leftarrow (\emptyset, \emptyset \rightarrow \mathbb{R})$  {Initial guess, e.g., all zeros}
6:   Set counterexample_flag  $\leftarrow$  False
7: while not counterexample_flag do
8:    $b_i, \dots, b_K \leftarrow \text{allocate}(\mathcal{S}, \tilde{\theta}_1, \tilde{\theta}_2, \dots, \tilde{\theta}_K)$  {Allocate bundles to maximize social welfare via integer programming}
9:   counterexample_flag  $\leftarrow$  False
10:  for each proxy  $i = 1$  to  $K$  do
11:     $\phi_i(b_i) \leftarrow \text{compute\_lindahl\_prices}(\phi, b_i)$  {Determine Lindahl prices for bundle  $b_i$ }
12:     $(r, \theta') \leftarrow \mathcal{M}_C^{p'_i}(\phi_i, b_i)$  {Check if satisfied}
13:    if  $r = 0$  then
14:       $\theta_i \leftarrow \theta'$ 
15:      counterexample_flag  $\leftarrow$  True {Need to reallocate}
16:    end if
17:  end for
18: end while
19: return  $\{(b_i, \phi_i(b_i)) \mid i = 1, 2, \dots, K\}$  {Allocated bundles and corresponding prices}

```

Table 1: Scenarios and Available Items

Scenario	Items		Description
Electronics	1. Apple AirPods (2nd Generation) 3. Apple iPad (9th Generation) 5. Apple Pencil (2nd Generation)	2. Apple AirPods Max 4. Apple iPad Air (M2) 6. Apple Pencil Pro	A collection of like-new electronics donated to a local library, ranging from AirPods to iPads and accessories.
Preserves	1. Organic Strawberry Jam 3. Apricot and Lavender Conserve 5. Spiced Plum Chutney	2. Wild Blueberry Preserves 4. Sugar-Free Raspberry Spread 6. Tropical Mango and Passionfruit Jam	Gourmet food items are available, including exotic jams and chutneys with unique flavors, perfect for cooking or gourmet meals.
Transportation	1. Electric Scooter S2 Pro (2024) 3. Voltron SP03 Electric Scooter (2024) 5. Titan Escape 3 (2023)	2. Electric Scooter Elettrica (2023) 4. Troik Verve+ 2 (2023) 6. Schwin Suburban (2021)	A variety of electric scooters and bikes are up for auction, catering to urban commuters and fitness enthusiasts.

4 LLM Pipelines for Bidder Simulation

We simulate the population of people that will be bidding in the auction using LLM-based pipelines. We use LLMs to construct the people’s preferences in a way that is natural and realistic and in a way where they can answer natural language questions about their preferences.

The definition of preferences as the seed $\mathcal{D}(p)$ is crucial for the simulated person to be able to answer natural language questions about their preferences. Indeed, if simulated preferences arise from arbitrary orderings of bundles this, first of all, does not necessarily represent realistic preferences, and second of all, cannot be meaningfully described in natural language.

4.1 Making LLM calls

When we describe making a call to an LLM, the following three steps are performed:

1. *Collate* information the relevant information into a prompt
2. *Call* an LLM with the prompt. The prompts are then passed to OpenAI’s gpt-4o-mini model OpenAI [2024], versioned July 18, 2024. Unless otherwise specified, all LLM calls in this work use this model.
3. *Compile* the response from the output of the LLM. The responses are either extracted using regular expressions from the LLM output or using the structured outputs feature. If the responses cannot be identified as such or the LLM otherwise errors in some way, we repeatedly call the LLM with the same prompt until it succeeds.

4.2 Seed response LLM pipeline

The process for generating the seeds, $\mathcal{D}(p)$, which define person p ’s preferences is done in a four-step LLM pipeline.

The first step of the pipeline generates the germ of the seed. Each subsequent step revises the seed to be more specific and more coherent. We take the final revised preferences at the end of step 4 to be the true preferences.

Step 1: Initial Preference Generation. The LLM generates a person’s specific purchase occasion and taste over the items, which forms an initial description of a person’s preferences based on the given auction scenario. To encourage diversity in responses, we ask the LLM to generate three possible versions and then randomly sample one.

Step 2: Revision for Clarity and Consistency. The LLM revises the initial preferences to enhance clarity and internal consistency in describing the person’s preferences.

Step 3: Anchor with Specific Valuations. The LLM anchors the person’s preferences by assigning explicit dollar values to specific bundles of interest.

Step 4: Refinement for Precision in Complex Bundles. The LLM refines the person’s preferences to outline a process that defines how they evaluate other unspecified bundles, considering items as substitutes and complements.

4.3 Message response LLM-based pipeline

In order to ensure that the simulated person acts in a coherent manner consistent with a standard model of preferences, at the outset, the pipeline constructs an XOR valuation $\theta = (B, v)$ for the person. θ is cached. The pipeline then responds to value and demand queries using θ . We implement two subroutines to perform LLM calls. The first subroutine is used to help construct θ . The second subroutine is used to respond to questions.

The first subroutine takes as input a bundle and gives an *initial* value for the bundle as output. Maintaining a working valuation $\theta = (B, v)$, we iterate over an enumeration of all possible bundles in ascending order of the number of items they contain. At each iteration, we run the first subroutine to

get the *initial* value ν of the bundle and update θ with that bundle and its value if free disposal is not violated, e.g. if $v^*(b) < \nu$.

The second subroutine takes as input a question and gives an answer as output. This answer is directly given to the proxy. The prompts for the LLM calls are below.

4.3.1 LLM subroutines

Question prompt:

```
Description of scenario:  $\mathcal{D}(\mathcal{S})$ 
Description of person:  $\mathcal{D}(p)$ 
*****
The person is asked the following question:  $q$ 
*****
Please reason through and then craft how you think the person would respond to
the question. Keep the response concise (only one sentence) and relevant to
the question.
```

Value query prompt:

```
Description of scenario:  $\mathcal{D}(\mathcal{S})$ 
Description of the person:  $\mathcal{D}(p)$ 
*****
The person has the option to receive bundle:  $\mathcal{D}(b)$ 
*****
Please determine the value the person assigns to this bundle. Provide the maximum
dollar amount they are willing to pay in the format.
```

5 LLM Pipelines for Proxies

We implement various proxy designs that incorporate LLMs into proxies. Each proxy design ω must implement $\mathcal{M}_C^{\omega(p)}$. The proxy takes as input the prices ϕ and allocated bundle b and gives as output either $(1, v)$ indicating that the person is satisfied with b at ϕ , or $(0, v')$ indicating that the person is not satisfied and updates their manifest valuation to v' , where v, v' are XOR valuations.

5.1 Drop-in LLM proxy

Drop-in LLM proxies are those that only use value and demand queries when messaging their person. They are drop-in replacements for proper-learning proxies like ω_{xor} because they have message their person with the same message types: value queries and demand queries. We give two such designs.

Design 1 $\omega_{VD,1}$ maintains in its memory a transcript of the conversation c_i (with i being the given iteration of the auction) and a hypothesis XOR valuation $\theta = (B, v)$. For person p and its proxy $p' = \omega_{VD,1}(p)$, the algorithm implements $\mathcal{M}_C^{p'}$ as the algorithm, which when called with prices ϕ and allocated bundle b , prompts an LLM with $\mathcal{S}$, ϕ , b , and c_i where the LLM is asked to choose one of the following actions:

1. message the person a value query;
2. message the person a demand query; or
3. reply to the auction that the person is satisfied.

With (1), the LLM also specifies a bundle b with which we call $\mathcal{M}_V^p(b)$ to message the person to receive their value ν for the bundle which is used to create an updated hypothesis valuation $\theta' = (B', v')$, where $B' = B \cup \{b\}$ and v' is equal to v when given any item from B , and is equal to x when given b .

With (2), we call $\mathcal{M}_D^p(\phi, b)$ to get the person's demanded bundle b' . If $b' = b$, the proxy forwards along that the person is happy. If it is not, we call $\mathcal{M}_V^p(b')$ receiving the person's value ν of b' and θ is correspondingly updated.

With (3), the bidder simply returns that it's happy, i.e. $(1, \theta)$.

Design 2 $\omega_{VD,2}$ is similar to $\omega_{VD,1}$ except that instead of directly returning its updated hypothesis valuation $\theta' = (B', v')$ it computes an inferred valuation $\theta^* = (B^*, v^*)$, which gives the same value $v^*(b') = v'(b')$ for all $b' \in B'$ and additionally infers values $v^*(b^*)$ for all $b^* \in B'^c$, where B'^c is the set of bundles without specified values.

We compute θ^* using an inference function $\gamma : \mathcal{B} \rightarrow \mathbb{R}_+$ that gives an inferred value for each bundle. To do this, we follow the following procedure: maintaining a working $\theta^* = (B^*, v^*)$ initialized to θ' , we iterate over an enumeration of all possible bundles in ascending order of the number of items they contain and for each bundle β , if $\beta \notin B'$ and $v^*(\beta) < \gamma(\beta)$, we add β to B^* and define $v^*(\beta) = \gamma(\beta)$.

γ is calculated at scheduled iterations of the CECA auction, including includes the first iteration, using an LLM pipeline. The LLM pipeline, given the proxy's memory, c_i , of their conversation with p , the scenario $\mathcal{S}$, and a bundle β , infers a value ν for β . Thus γ can be constructed by calling the pipeline with all $\beta \in B'^c$. We discount the value given by the LLM by a constant factor of $\epsilon = 0.75$ as it is better for the person to under-represent their valuation because otherwise their Lindahl prices under the auction will be set artificially high.

The LLM pipeline to infer the bidders valuation queries for all bundles $b^* \in B'^c$ first collates a prompt of the form

Here is the scenario description : $\mathcal{D}(\mathcal{S})$

Here is a transcript of the conversation with the person: c_i

What do you think the person would value the following bundle at? $\mathcal{D}(b^*)$

and then calls an LLM with this prompt and compiles the bundle's inferred value from the LLM's response.

5.2 Plus-questions LLM proxy

The plus-questions proxy ω_{NVD} uses natural language questions in addition to value queries and demand queries when messaging their person.

$\mathcal{M}_C^{\omega_{NVD}(p)}$ is the algorithm that is identical to $\mathcal{M}_C^{\omega_{VD,2}(p)}$ except that the very first time it is run, it first calls $\mathcal{M}_N^p$ with question q where q is given by the LLM response to a prompt asking, "What should the proxy ask the person next to better understand their preferences?" given $\mathcal{D}(\mathcal{S})$.

Note that ω_{NVD} , like $\omega_{VD,2}$, also gives inferred θ' . The additional information p provides in response to q is recorded in c_i and thus is used to give more information to the LLM pipelines which choose what action to take and which infer the valuation p holds for bundles without specified values.

5.3 Hybrid XOR-LLM proxy

The hybrid XOR-LLM proxy ω_H combines ω_{NVD} and the ω_{xor} . For person p and proxy $p' = \omega_H(p)$, we define $\mathcal{M}_C^{p'}$ as follows .

At the start when $\mathcal{M}_C^{p'}$ is called we run $\mathcal{M}_C^{\omega_{NVD}(p)}$ such that the proxy is asking natural language questions and using the LLMs to infer valuations.

After a set number of iterations, α , we switch the algorithm such that when $\mathcal{M}_C^{p'}$ is called we run $\mathcal{M}_C^{\omega_{xor}(p)}$ where the proxy is now using the proper-learning algorithm. Throughout both the first α iterations and the period onwards, we continuously maintain the manifest valuation θ .

Furthermore before the switchover, recall that with $\mathcal{M}_C^{\omega_{xor}(p)}$ we periodically calculate the inference function γ and at each iteration, calculate and return the inferred valuation θ^* . We continue this after the switchover, calculating and returning θ^* based on the outputted θ given by $\mathcal{M}_C^{\omega_{xor}(p)}$. Furthermore, we specify that every iteration after the switchover, we update γ such that the new $\gamma'(b) = \delta \cdot \gamma(b)$ for all $b \in \mathcal{B}$. This has the purpose of maintaining, in the short run, the benefits of using an inferred valuation, but in the long run, removing the inferred valuation by decaying γ towards 0.

6 Results

The auction, proxies, and simulated bidders were implemented in Python. We used Google’s OR Tools package for integer programming for the auction. We use OpenAI’s gpt-4o-mini model, versioned to July 18, 2024, unless otherwise stated.

For each of the three scenarios outlined in Table 1, we generated three tranches of three seeds each, for a total of 27 simulated participants in the auctions.

6.1 Robustness of LLM-Based Bidder Modeling

We found that the seeds generated by the seed-generation pipeline are robust. To understand if the seeds robustly describe a specific valuation, we examined the inter-model robustness of the seeds, comparing the valuations generated by the message response LLM-pipeline using the gpt-4o-mini model to those generated with the gemini-1.5-flash model DeepMind [2024], we found that the valuations were fairly consistent as shown in Figure 2. Because the models are provided by separate teams from OpenAI and Google, respectively, this shows that the seeds and the valuations interpreted from it via the message-response pipeline are not specific to a specific family of models and thus generally interpretable and robust.

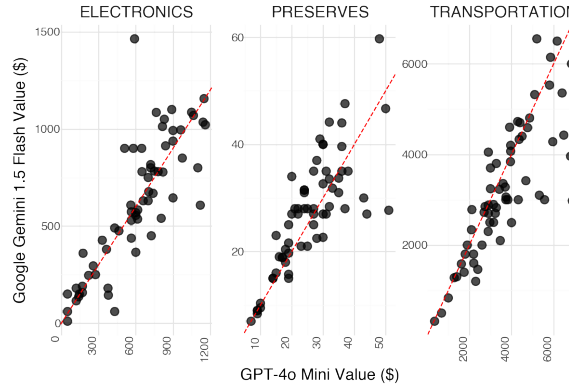


Figure 2: Comparison between GPT-based and Gemini-based models in bidder robustness tests. Gpt-4o-mini and gemini-1.5-flash give similar values in response to value queries evidenced by the dots, each of which representing a single bundle, falling near the identity gpt-4o-mini value (\$) = gemini-1.5-flash value line (\$).

6.2 Variability and Shape of Preferences

Throughout the seed generation process, we are agnostic to the valuation function implied by the seed. Thus, we ran a set of experiments to understand the shape of valuations implied by the seed.

For a specific seed in each scenario, we ran a titration, shown in Figure 3, where starting from an empty bundle, we incrementally added items to the bundle and then incrementally removed items from the bundle. At each step, we ran the message response LLM-pipeline for value queries ten times as similarly to the previous section test for robustness — this time intra-model robustness.

We found that the valuation given were precise. The variation for a given bundle is tight and the overall shape satisfies a few characteristics. First, the preferences are monotonic or, equivalently put, they satisfy free disposal. We can see in Figure 3 that as we add items to the bundle the valuation of that bundle consistently goes up and, similarly, as we remove items from the bundle the valuation of the bundle consistently goes down.

6.3 Auction Efficiency

We find through simulation that we can reach more efficient outcomes more quickly using LLM-based proxies. Quantitatively, we can reach 75% efficiency in ten interactions between the person and their proxy with ω_{xor} , in four interactions with $\omega_{VD,2}$ and in two interactions with ω_{NVD} (Figure 4).

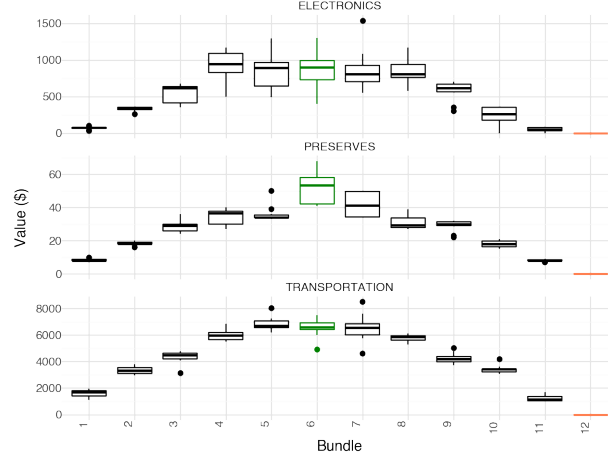


Figure 3: Preference variability and shape across repeated value queries to a single person. Each boxplot displays the distribution of values for a single bundle ($n = 10$). Bundle 6, in green, has all 6 items. Bundle 12, in orange, has no items. Bundle $i \leq 6$ has items $T_1, \dots, T_i$. Bundle $6 < i \leq 12$ has items $T_i, \dots, T_6$.

For the nine tranches of three bidders each with three tranches per scenario, we ran the auction using either ω_{xor} , $\omega_{VD,1}$, $\omega_{VD,2}$, or ω_{NVD} to represent all of the bidders, halting the simulation after 16 iterations. We fully elicit each of the 27 people’s preference across all 2^6 bundles and use that to calculate to welfare-maximizing outcome.

Each iteration of the competitive equilibrium auction, we calculate the social welfare of the auction and record the number of interactions between each person and their proxy. We then calculate for each iteration the percent efficiency by dividing the social welfare by that of the social welfare maximizing outcome. We also calculate the average number of interactions. We then impute for $i = 1, 2, \dots, 16$ the efficiency with i interactions by selecting the efficiency at the iteration of the run that has the lowest number of interaction greater than i . Then for each i we average over all nine tranches to get the percent average efficiency which is shown in Figure 4.

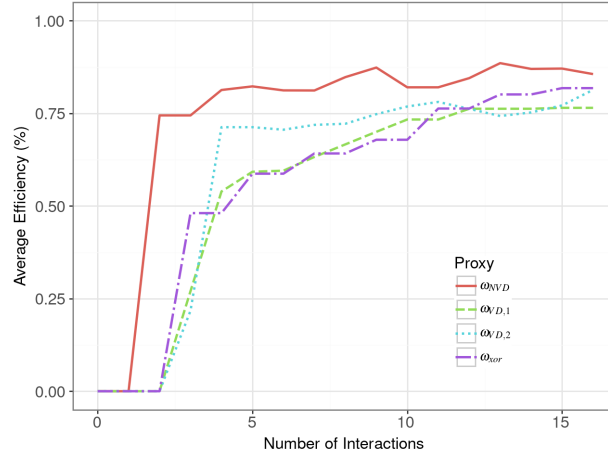


Figure 4: Average percent efficiency of the auction run under the three scenarios with three people participating in each. Anchoring on our XOR elicitation proxy, we see that the drop-in proxy $\omega_{VD,1}$ using an LLM performs similarly, both reaching 75% efficiency in around 10 interactions between person and proxy. The plus-questions proxy $\omega_{VD,2}$ performs markedly better, reaching the 75% of efficiency in four interactions that otherwise would take ten interactions. We see that proxy ω_{NVD} that is also allowed to use natural language questions reaches the same 75% efficiency in two interactions.

6.4 Long run performance of hybrid XOR-LLM proxy

The previous section showed how in the short run, LLM-based proxies outpace the proper-learning-based ω_{xor} . We now consider the hybrid XOR-LLM proxy ω_H to characterize the long-term performance of an LLM-based proxy, showing that we have both the short term rapid increase in efficiency without sacrificing long term ability of the proxy to exactly learn the person’s valuation and of the auction using the proxy to converge to the social-welfare-maximizing outcome. For the following experiments, we set $\alpha = 10, \delta = 0.95$.

First, we ran ω_H and ω_{xor} as preference learning algorithms, by running the auction with only one person with either ω_H or ω_{xor} as a proxy and constantly giving prices equal to the manifest valuation and allocating the empty bundle, and measured at each step the number of times the proxy had interacted with the person and the average error, calculated as the average absolute difference between the value of an item calculated using the person’s XOR valuation and the value of an item calculated using the proxy’s manifest valuation. These numbers are aggregated over the multiple tranches using the methodology in Section 6.3.

The results, plotted in Figure 5(a), show that ω_H approximates the person’s valuation significantly quicker as the line for ω_H is significantly and consistently lower than that for ω_{xor} . To put numbers to it, in all three scenarios, ω_H gets to a better approximation of the person’s valuation than ω_{xor} does in 50. We must note however, that at the very end, ω_{xor} gets to 0 error, e.g. exactly learns the person’s valuation, around 10 iterations quicker. This is because the *LLM* is sometimes inefficient with its queries and because, though helpful for approximations, natural language questions don’t help as much for exactly learning.

Second, we ran the auction simulations identical to those in Section 6.3, except that we let the auction continue running until completion. We aggregate and plot the data according to the same procedure and plot the results in Figure 5(b).

We see that in the initial phase, ω_H rapidly reaches fairly efficient outcomes. This matches the results shown for ω_{NVD} which is expected as literally $\mathcal{M}_C^{\omega_H(p)} = \mathcal{M}_C^{\omega_{NVD}(p)}$ for the first α iterations. We also see that the efficiency of the outcome steadily increases as we increase the number of interactions, in line with ω_{xor} such that the social-welfare-maximizing outcome is also reached.

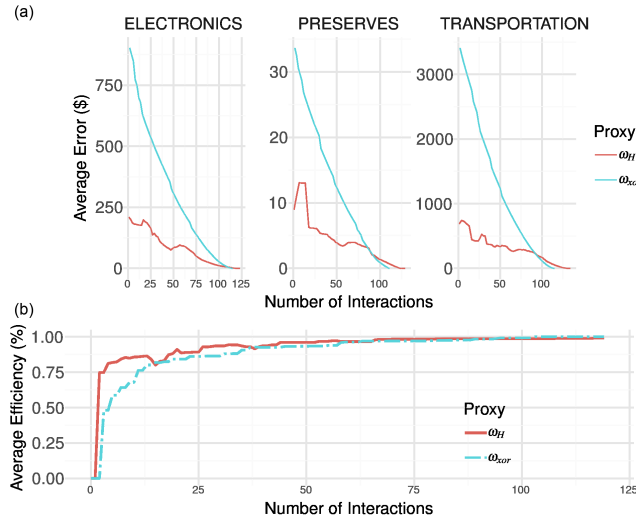


Figure 5: Performance of the hybrid XOR-LLM proxy ω_H (in red) and the XOR elicitation proxy ω_{xor} (in teal) under varying numbers of interactions with the person they represent. Panel (a) shows that ω_H gets to a good approximation of the person’s valuation significantly quicker versus ω_{xor} while still converging to the person’s exact valuation. Panel (b) shows that when running the auction with ω_H , similar to Figure 4 we get a sharp increase in efficiency in the initial stages, while still converging on the welfare-maximizing outcome in the long run.

7 Conclusion

In this study, we presented an innovative approach to preference elicitation in combinatorial auctions by integrating Large Language Models (LLMs) as proxies. Traditional query-based elicitation methods, while theoretically efficient, often impose substantial cognitive burdens on bidders due to the extensive communication required. Our LLM-based proxy design mitigates this issue by leveraging natural language interactions, thereby streamlining the elicitation process and significantly reducing the number of necessary queries.

Empirical evaluations demonstrate that LLM-based proxies, particularly those that use LLMs to infer the bidder’s valuation on unspecified bundles, achieve approximately efficient auction outcomes up to five times faster than classical XOR elicitation proxies. As illustrated in Figure 4, proxies such as $\omega_{VD,2}$ and ω_{NVD} reach 75% of maximum social welfare with markedly fewer interactions compared to the ω_{xor} proxy. Furthermore, the hybrid XOR-LLM proxy ω_H not only accelerates the initial elicitation phase but also converges to the social-welfare-maximizing outcome, suggesting a path forward for combining the rapid approximation capabilities of LLMs with the guarantees of classical preference elicitation methods.

Additionally, the robustness of LLM-based bidder modeling was validated across different models and repeated trials, ensuring consistent and reliable preference representations. This robustness underscores the feasibility of employing natural language processing in complex auction settings, enhancing both the theoretical and practical aspects of auction design.

In summary, our findings highlight the potential of LLMs to revolutionize preference elicitation in multiagent systems by making the process more intuitive via natural language questions and less burdensome via significantly fewer queries for participants. Future research could explore the scalability of this approach to larger auction environments and investigate the integration of the increasingly advanced LLM capabilities to further enhance elicitation efficiency and accuracy for lower cost. The promising results presented herein pave the way for more human-centric and efficient auction mechanisms in increasingly complex market landscapes.

References

- A. Blum, J. Jackson, T. Sandholm, and M. Zinkevich. Preference elicitation and query learning. *J. Mach. Learn. Res.*, 5:649–667, Dec. 2004. ISSN 1532-4435.
- D. Braziunas. *Computational Approaches to Preference Elicitation*. PhD thesis, University of Toronto, Toronto, 2006. URL <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=7caa3033711f60f1a63a6b42bf375c8df00ccd42>.
- E. Budish and J. B. Kessler. Can market participants report their preferences accurately (enough)? *Management Science*, 68(2):1107–1130, 2022.
- P. C. Cramton. The FCC Spectrum Auctions: An Early Assessment. 1997.
- G. DeepMind. Gemini breaks new ground with a faster model, longer context, AI agents and more, May 2024. URL <https://blog.google/technology/ai/google-gemini-update-flash-ai-assistant-io-2024/>.
- H. Ergin and T. Sarver. A unique costly contemplation representation. *Econometrica*, 78(4):1285–1339, 2010.
- J. J. Horton. Large Language Models as Simulated Economic Agents: What Can We Learn from Homo Silicus? 2023.
- P. Klemperer. *Auctions: theory and practice*. Toulouse lectures in economics. Princeton University Press, Princeton, 2004. ISBN 978-0-691-11426-2 978-0-691-11925-0. OCLC: ocm53001276.
- S. M. Lahaie and D. C. Parkes. Applying learning algorithms to preference elicitation. In *Proceedings of the 5th ACM Conference on Electronic Commerce*, pages 180–188, 2004.

- K. Nielsen and J. Rehbeck. When Choices Are Mistakes. *American Economic Review*, 112(7):2237–2268, July 2022. ISSN 0002-8282. doi: 10.1257/aer.20201550. URL <https://pubs.aeaweb.org/doi/10.1257/aer.20201550>.
- N. Nisan and I. Segal. The communication requirements of efficient allocations and supporting prices. *Journal of Economic Theory*, 129(1):192–224, July 2006. ISSN 00220531. doi: 10.1016/j.jet.2004.10.007. URL <https://linkinghub.elsevier.com/retrieve/pii/S0022053105000311>.
- OpenAI. GPT-4o mini: advancing cost-efficient intelligence, 2024. URL <https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/>.
- I. Palacios-Huerta, D. C. Parkes, and R. Steinberg. Combinatorial Auctions in Practice. *Journal of Economic Literature*, 62(2):517–553, June 2024. ISSN 0022-0515, 2328-8175. doi: 10.1257/jel.20221679. URL <https://pubs.aeaweb.org/doi/10.1257/jel.20221679>.
- D. C. Parkes. Auction design with costly preference elicitation. *Annals of Mathematics and Artificial Intelligence*, 44:269–302, 2005.